

BRAWL STARS BALANCE ANALYZER

A Technical Report on Data Engineering,
Feature Design, and Machine Learning

Hudson Kimm

Applied Mathematics and Computer Science

UNC-Chapel Hill

July 2026

103 brawlers | 107 columns | 70+ engineered attributes | 5-fold CV

Executive Summary

Brawl Stars provides an interesting machine learning case study because competitive strength emerges from the interaction of dozens of gameplay mechanics, making it a natural testbed for feature engineering and predictive modeling.

This project began with a deceptively simple question: can the competitive strength of a Brawl Stars brawler be estimated from the way that brawler is designed? Answering that question required substantially more than training a model. No single ready-to-use dataset contained both rank-level performance and the design variables needed for prediction, so the project evolved into an end-to-end data and machine learning system.

The work progressed through five major stages: defining a meaningful prediction target; sourcing, cleaning, and aligning multiple datasets; engineering domain-specific gameplay features; using exploratory analysis to understand the data; and iteratively improving predictive models through validation, tuning, ablation, and feature selection.

Final outcome

A tuned Random Forest using six RFECV-selected gameplay features achieved the strongest benchmark result: mean cross-validated MAE 1.476 and mean R^2 0.137. The most influential selected features were Survivability, Decision Complexity, and Skill Ceiling.

The project's most important conclusion is not that one algorithm solved the problem. The largest improvements came from improving the representation of the problem: building a cleaner dataset, designing better features, and removing redundant information. Static design characteristics contain meaningful signal, but they cannot fully explain a live competitive meta shaped by maps, team composition, player behavior, and balance updates.

Report Roadmap

1. Define the prediction problem and choose what to measure.
2. Construct and validate a unified dataset.
3. Translate game mechanics into model-ready features.
4. Use exploratory analysis to test assumptions and identify limitations.
5. Build baseline models and diagnose failure.
6. Improve the model through tuning and feature analysis.
7. Benchmark alternative model families under a consistent protocol.
8. Interpret the final model and define practical next steps.

1. Framing the Problem

1.1 The central question

The project was designed to test whether competitive performance can be predicted before observing a brawler in the live meta. In practical terms, this means using design characteristics like health, damage pattern, range, mobility, utility, mechanical difficulty, and similar attributes to estimate how a brawler is likely to perform.

This is more difficult than predicting a static property. Competitive performance is an outcome of interaction: the brawler, the player, the map, the mode, teammates, opponents, and the current balance patch all contribute. The project therefore treats the model as an explanatory and comparative tool rather than a perfect forecasting system.

1.2 Choosing the target

Several targets were possible. Win rate is intuitive but can be distorted by selection effects. Use rate measures popularity more than strength. A single overall rank collapses too much information. The project ultimately focused on MetaScoreScaling, a derived target intended to capture how a brawler’s Meta Score changes across ranked tiers.

Candidate target	Strength	Limitation
Win Rate	Direct performance measure	Can be inflated or depressed by who chooses the brawler
Use Rate	Captures competitive adoption	Popularity is not the same as strength
Meta Score	Combines performance and usage context	Still reflects the current environment
MetaScoreScaling	Captures rank-dependent change	Requires consistent performance data across tiers

Selecting a rank-sensitive target aligned the modeling task with the project’s broader question: which brawler designs become more or less effective as player skill and competitive difficulty increase?

1.3 Defining success

Success was defined using repeated out-of-sample evaluation rather than a single favorable split. Mean Absolute Error (MAE) was the primary metric because it is directly interpretable in target units and less sensitive to extreme errors than squared-error metrics. R^2 was retained as a secondary measure of explained variance.

2. Building the Data Foundation

2.1 Why data engineering became the first major challenge

The initial plan assumed that gameplay and competitive statistics could be analyzed together immediately. In practice, the relevant information was distributed across multiple sources and schemas. Performance data existed by rank, while gameplay attributes and manually labeled mechanics lived in separate tables. The first substantial milestone was therefore not modeling; it was constructing a reliable master dataset.

2.2 Collection, cleaning, and alignment

- Collected rank-level Win Rate, Use Rate, and Meta Score data.
- Cleaned names and validated one-to-one alignment across source tables.
- Removed the non-comparable Bolt entry to preserve dataset consistency.
- Added core statistics to the engineered feature table.
- Merged feature and performance tables into data/master_dataset.csv.
- Checked required columns, duplicates, missing values, and row alignment.

The final master dataset contains 103 brawlers and 107 columns. The validation work was critical: a model can run on a misaligned dataset while producing plausible-looking but meaningless results. Explicit name validation and duplicate checks reduced this risk.

2.3 Derived performance measures

Rank-specific data were supplemented with aggregate and variability measures. These included averages, standard deviations, ranges, and scaling variables for Meta Score, Win Rate, and Use Rate. The derived measures allowed the project to distinguish overall strength from consistency and rank-dependent behavior.

Data layer	Examples	Purpose
Rank-level performance	Win Rate, Use Rate, Meta Score by tier	Preserve competitive context
Aggregates	AverageMetaScore, AverageWinRate	Summarize overall performance
Variability	MetaScoreStdDev, WinRateRange	Measure stability across tiers
Scaling targets	MetaScoreScaling, WinRateScaling	Model progression with rank

3. Translating Game Design into Features

3.1 Why raw statistics were insufficient

Raw statistics such as health and damage are easy to collect, but they do not fully express how a brawler functions. Two brawlers with similar damage can differ dramatically in mobility, safety, burst timing, control, or mechanical difficulty. The project therefore introduced domain-specific features that describe gameplay behavior rather than only surface-level numbers.

3.2 Feature families

Feature family	Representative variables	What it captures
Core statistics	Health, TotalDamage, RangeScore, AmmoCount	Basic numerical design
Combat	ReloadSpeedScore, BurstDamageScore, SustainedDPSScore	Damage delivery and tempo
Gameplay	MobilityScore, SkillCeiling, SurvivabilityScore	Strategic and mechanical identity
Mechanics	HasDash, HasHealing, HasStun, HasWallBreak	Presence of discrete abilities
Categories	Class, Rarity, RangeClass, SuperType	Broad archetypes and labels

3.3 Manual feature engineering

The gameplay feature group was the most judgment-intensive part of the dataset. Scores such as `DecisionComplexity`, `PositioningImportance`, `SkillFloor`, `SkillCeiling`, `TeamUtilityScore`, and `SurvivabilityScore` were designed to encode concepts that raw statistics miss. This introduced subjectivity, but it also allowed the model to represent the actual decision-making and skill expression that define competitive play.

Design principle

Each engineered feature was intended to describe a distinct gameplay concept. Later ablation experiments were used to test whether those concepts added predictive signal or merely introduced redundancy.

3.4 Reusable feature architecture

To keep experiments consistent, all feature groups were centralized in `feature_groups.py`. Every model, ablation study, and tuning script imported the same definitions. This eliminated duplicated lists and reduced the risk that two experiments would appear comparable while silently using different inputs.

4. Exploratory Analysis: Learning What the Data Could and Could Not Explain

4.1 Correlation analysis

Exploratory analysis began with relationships among core performance and gameplay variables. The correlation matrix showed that several intuitive raw attributes had weak linear relationships with performance. Use Rate and Meta Score were strongly related, while many design variables showed only modest correlations. This suggested early that simple linear modeling would likely be inadequate.

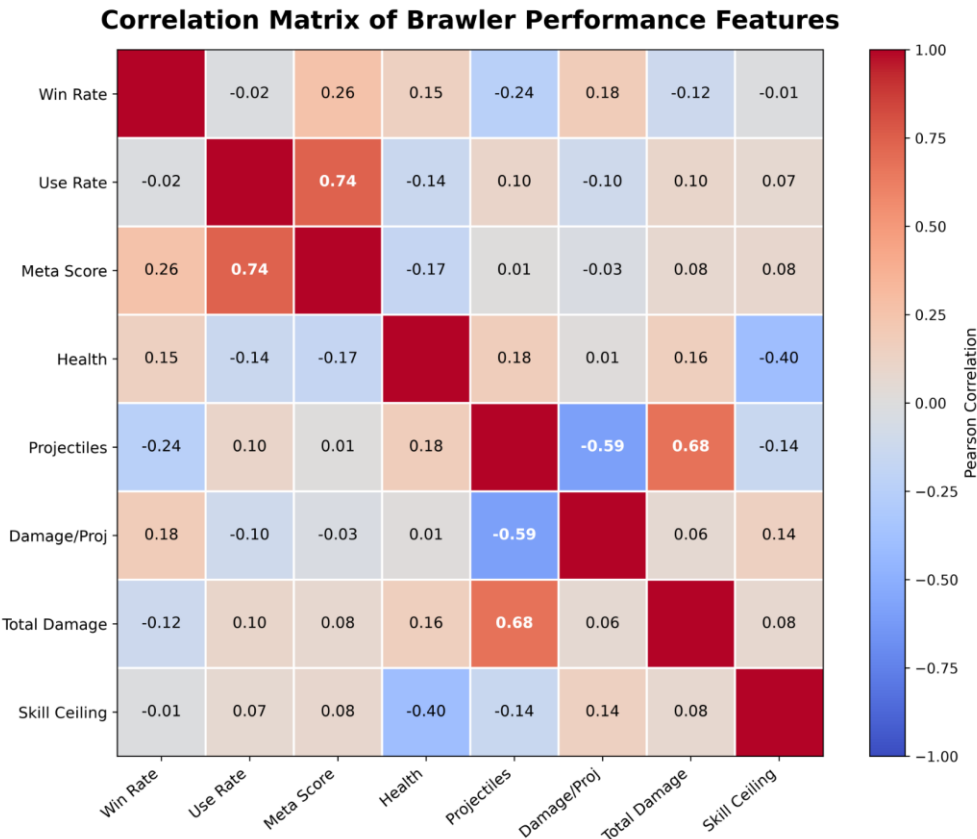


Figure 1. Correlation matrix for selected performance and gameplay variables. Strong pairwise relationships exist among some constructed or related variables, but most raw design features have limited linear association with performance.

4.2 Rank progression

The rank-progression visualization revealed that brawlers do not move uniformly through the competitive ladder. Some characters improve dramatically at higher ranks, while others decline. This supported the decision to model a scaling target rather than relying only on an overall average.

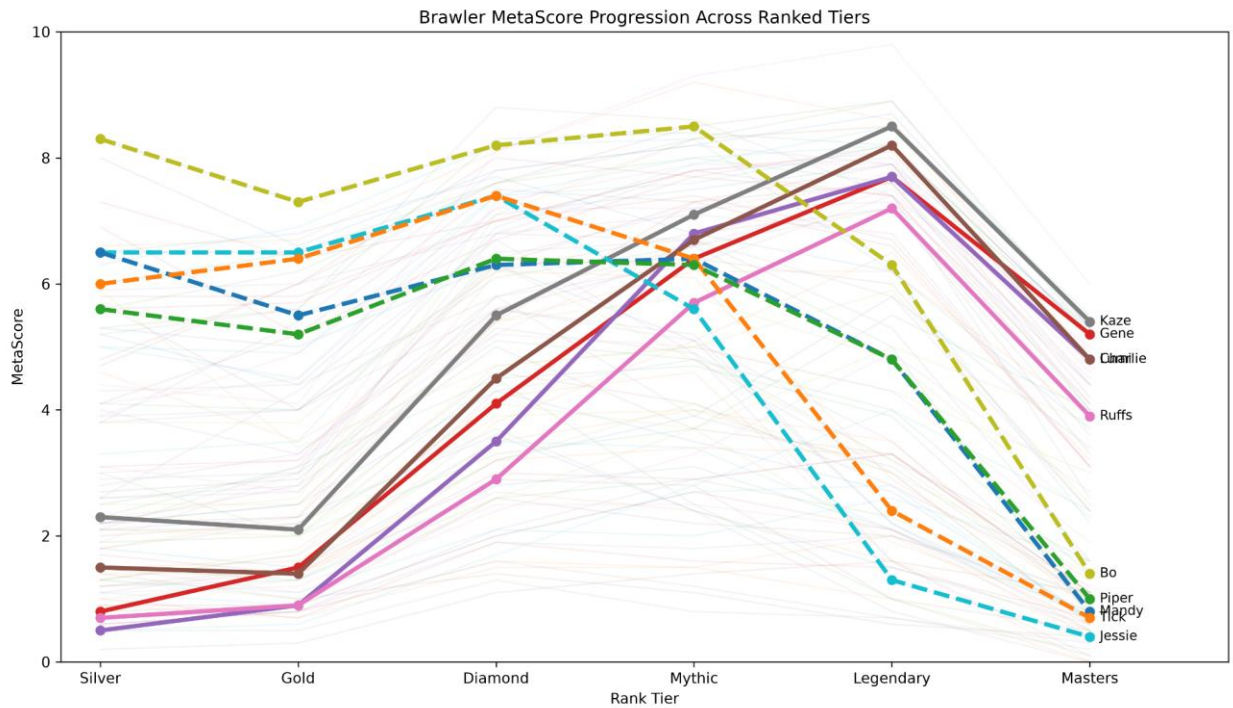


Figure 2. Meta Score progression across ranked tiers. Highlighted trajectories show that performance can change sharply with rank, indicating interaction between brawler design and player skill.

4.3 Extreme changes across tiers

Examining the largest positive and negative shifts provided a concrete view of what the target represented. Brawlers such as Charlie, Gene, Lumi, and Ruffs increased substantially from lower to higher tiers, while Jessie, Tick, Dynamike, and Bo declined. These patterns were not treated as universal balance judgments; they were evidence that rank-dependent performance is a meaningful phenomenon.

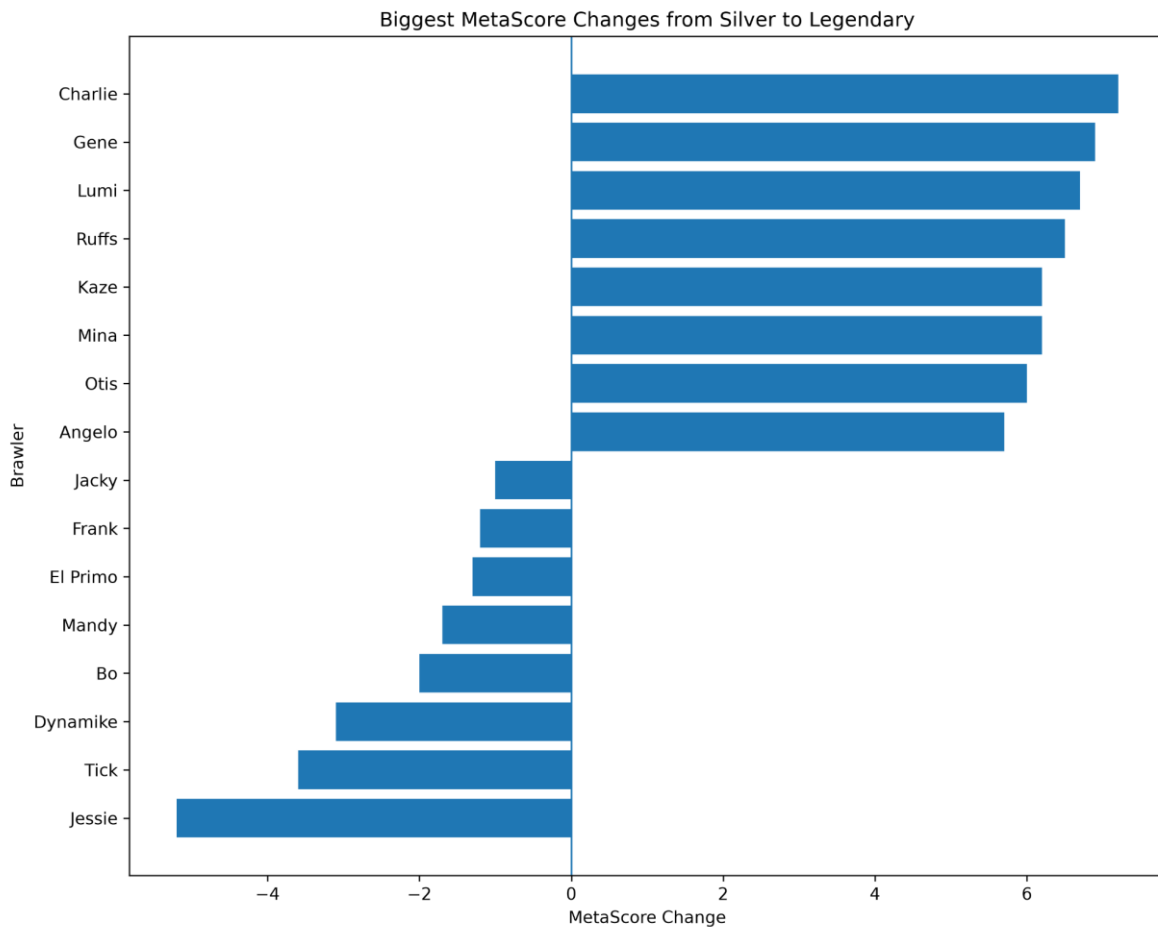


Figure 3. Largest Meta Score changes between Silver and Legendary. Positive values indicate improvement at higher tiers; negative values indicate decline.

4.4 What the exploratory phase changed

- Confirmed that rank progression was worth modeling.
- Showed that simple pairwise correlations were too weak to explain the target.
- Motivated nonlinear models and richer gameplay features.
- Revealed that static attributes would likely explain only part of the competitive environment.

5. First Models and the Value of Failure

5.1 Linear Regression as a diagnostic baseline

Linear Regression was intentionally used first because it provided a transparent baseline. Its poor performance, MAE 3.878 and R^2 approximately -2.08 in the original experiment, showed that the relationship between engineered design features and MetaScoreScaling was not well represented by a single linear combination.

This was not wasted work. The failed baseline narrowed the problem: either the available variables lacked sufficient signal, the relationship was nonlinear, or both.

5.2 Random Forest and the first improvement

A Random Forest Regressor was selected next because it can model nonlinear interactions without requiring large-scale feature transformations. The initial single-split result improved substantially, but later cross-

validation showed that the split was optimistic. This led to a methodological correction: five-fold cross-validation became the standard evaluation procedure for subsequent experiments.

5.3 Evaluation protocol evolved during the project

Important interpretation note

Early values were produced under different evaluation setups, so they illustrate project progression but are not all strictly apples-to-apples. Final model-family comparisons used the same six features and identical five-fold folds.

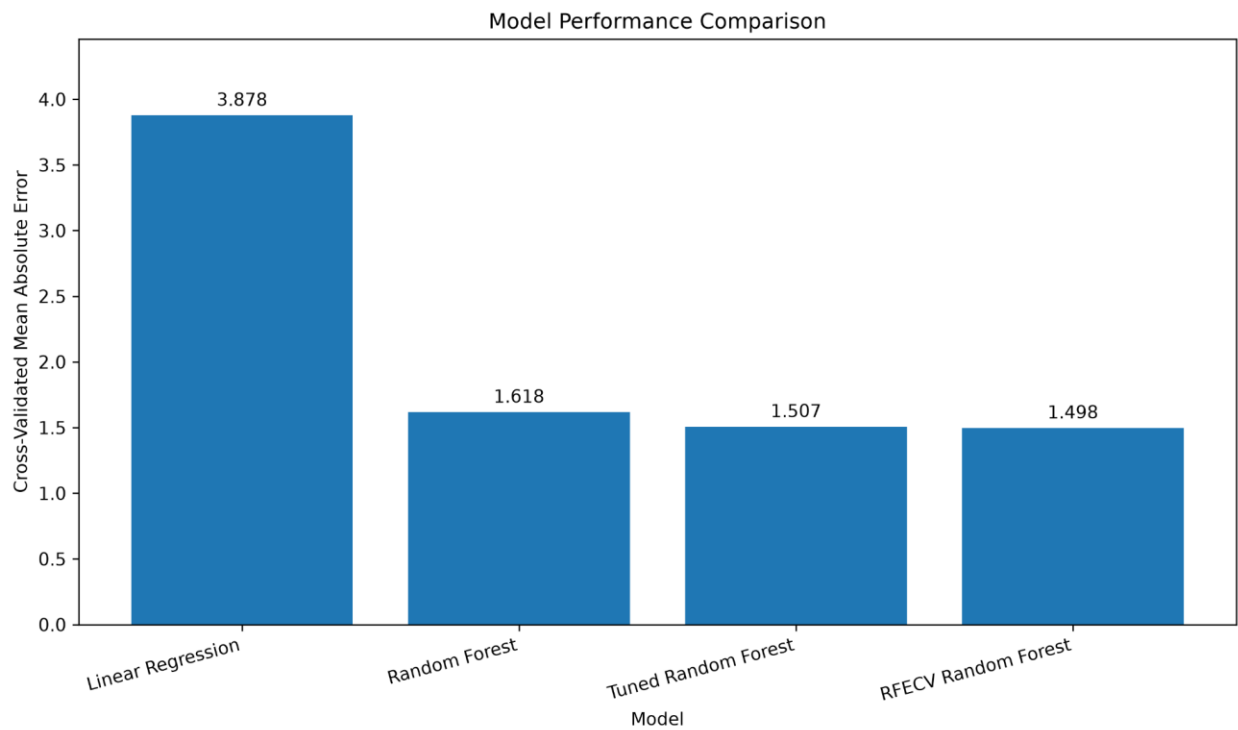


Figure 4. Model performance across the project’s iterative experiments. The largest gain came from moving beyond the linear baseline; later tuning and feature selection produced smaller improvements.

6. Systematic Model Improvement

6.1 Hyperparameter tuning

RandomizedSearchCV was used to search model complexity and sampling parameters. The best configuration used 500 trees, maximum depth 2, minimum split size 10, minimum leaf size 1, and log2 feature sampling. The shallow trees reflect the small dataset: stronger regularization reduced the risk of fitting noise.

Parameter	Selected value	Interpretation
n_estimators	500	Stabilize ensemble estimates
max_depth	2	Strongly limit tree complexity
min_samples_split	10	Require broader evidence before splitting
min_samples_leaf	1	Allow small terminal groups
max_features	log2	Decorrelate trees through feature subsampling

6.2 Feature-group ablation

Feature-group ablation tested Core Stats, Combat, Gameplay, Mechanics, Categories, combined subsets, and the complete feature set. The manually engineered Gameplay group performed best. The full feature set performed worse, indicating that additional variables introduced redundancy or noise rather than useful information.

Key lesson

More columns did not produce a better model. The quality and distinctiveness of the representation mattered more than the raw number of features.

6.3 Leave-one-feature-out analysis

Each of the 16 gameplay features was removed individually while the tuned Random Forest was reevaluated. Removing SurvivabilityScore increased MAE by 0.0405, the largest deterioration. DecisionComplexity, ObjectivePressureScore, PositioningImportance, MobilityScore, and TeamUtilityScore also showed useful signal. Several other removals slightly improved MAE, suggesting redundancy.

This analysis answered a local question (what happens when one feature is removed from the full gameplay set) but it did not identify the best combination. That limitation motivated recursive feature elimination.

6.4 Recursive Feature Elimination with Cross Validation

RFECV repeatedly removed lower-ranked features and evaluated the remaining subset. Using the same non-shuffled five-fold structure as the other selection experiments, the best point occurred with six features: MobilityScore, EscapeScore, PositioningImportance, DecisionComplexity, SkillCeiling, and SurvivabilityScore.

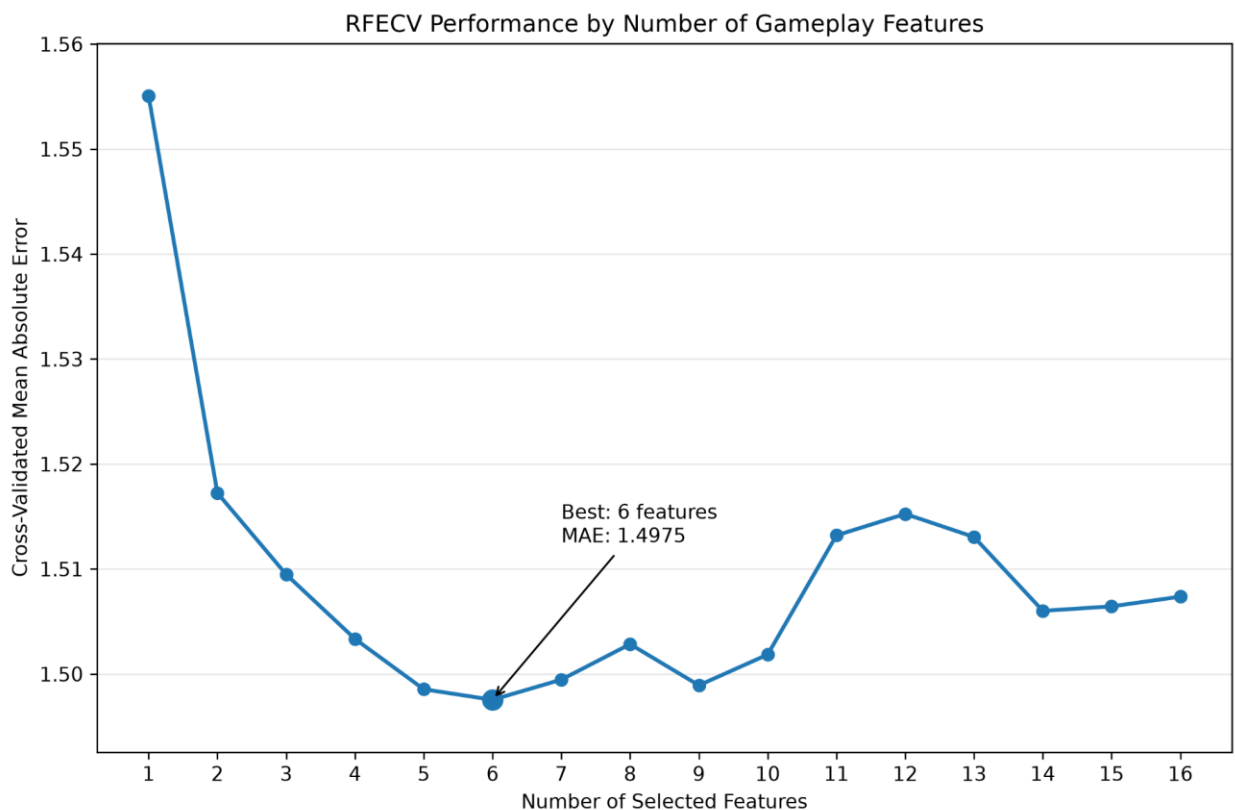


Figure 5. RFECV performance curve. The selection process reached its minimum MAE at six gameplay features, reducing the feature count by 62.5% relative to the original 16-feature gameplay set.

The improvement was modest, but the simplification was meaningful. A six-feature model is easier to interpret, less sensitive to redundant inputs, and more defensible than a larger model with nearly identical performance.

7. Benchmarking Alternative Ensemble Models

7.1 Why benchmark after feature selection

Model benchmarking was delayed until after feature selection so that the comparison would test model families rather than different data representations. Each benchmark used the same six selected features, the same target, the same five folds, and the same scoring metrics.

Model	Mean CV MAE	Mean CV R ²
Tuned Random Forest	1.476	0.137
Histogram Gradient Boosting	1.576	0.094
Extra Trees	1.741	-0.134
Gradient Boosting	1.744	-0.132

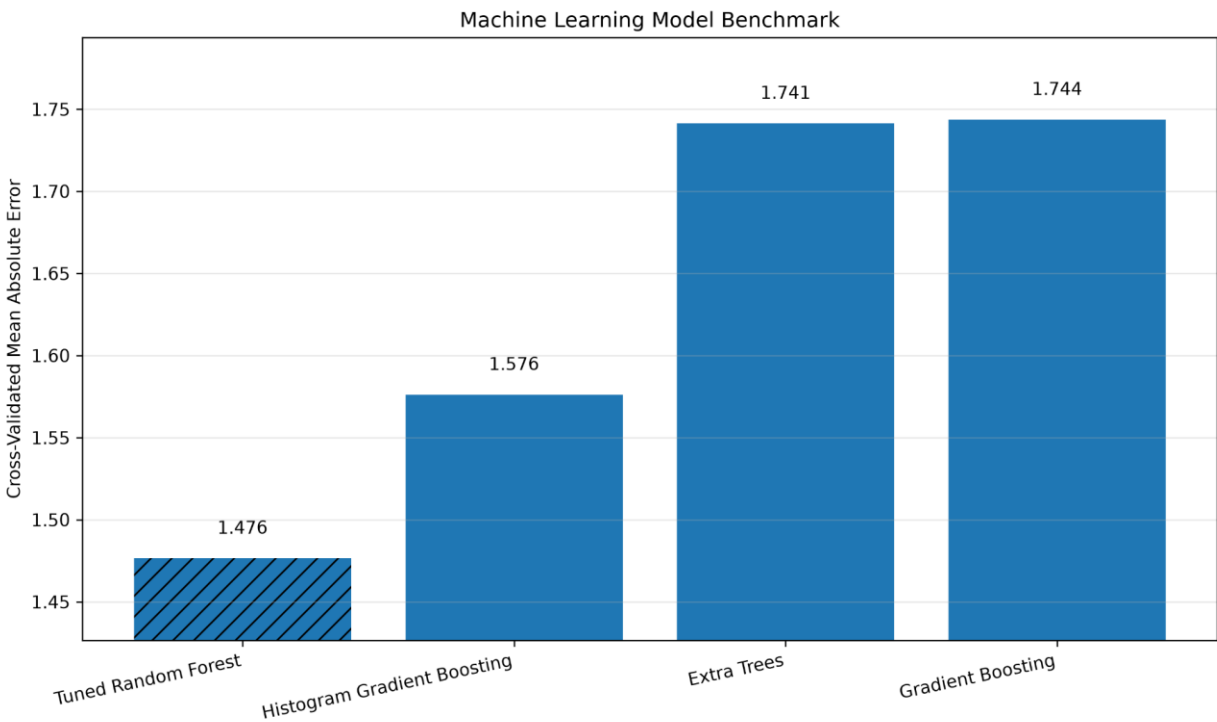


Figure 6. Final model benchmark under a consistent evaluation protocol. Lower MAE is better. The tuned Random Forest outperformed the three alternative ensemble methods.

The result supported keeping the tuned Random Forest as the final model. Histogram Gradient Boosting remained competitive but produced approximately 6.7% higher MAE. Extra Trees and default Gradient Boosting generalized poorly on this small, engineered dataset.

7.2 Why Random Forest likely worked best

- The dataset is small, making heavily regularized bagging attractive.

- The target likely depends on nonlinear thresholds and interactions.
- Shallow trees limited variance while retaining nonlinear structure.
- Feature subsampling reduced correlation among trees.
- Boosting methods may have been more sensitive to noise and sample size.

8. Interpreting the Final Model

8.1 Feature importance

The final Random Forest was fit using the six selected features. Impurity-based importance assigned the largest share to Survivability (0.294), followed by Decision Complexity (0.189), Skill Ceiling (0.156), Escape (0.153), Mobility (0.124), and Positioning (0.085).

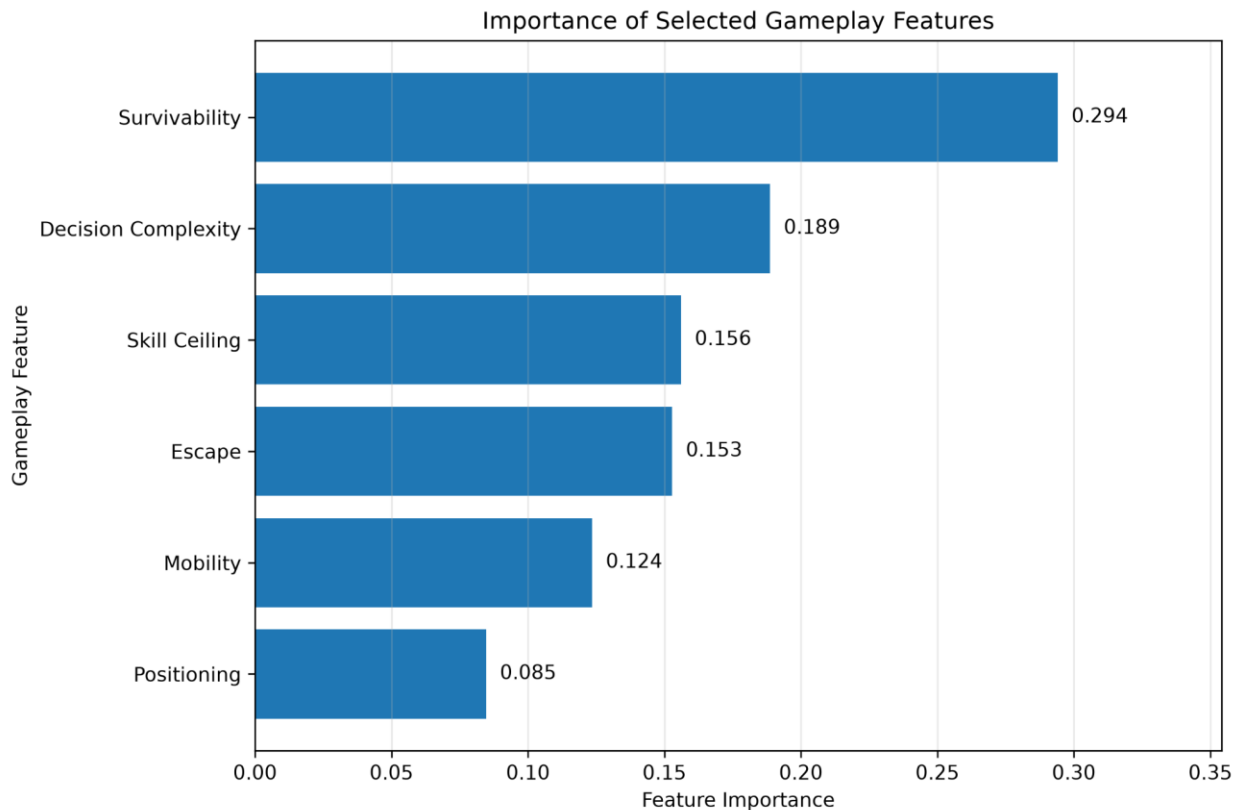


Figure 7. Feature importance within the final six-feature Random Forest. These values describe how the fitted model used the features; they do not establish causal effects.

8.2 Domain interpretation

The selected features form a coherent picture. Brawlers that survive pressure, create complex decisions, reward mastery, and possess reliable movement options are more likely to change meaningfully across skill tiers. Raw damage and health alone were less informative than the way those statistics combine into strategic flexibility and skill expression.

8.3 Model Limitations and What the Model Could Not Learn

Although the final Random Forest model consistently outperformed every other model evaluated, its performance also highlights the limitations of using only static gameplay characteristics to predict competitive strength.

The final model achieved a cross-validated MAE of 1.476 and an R^2 of approximately 0.14. While these results demonstrate meaningful predictive signal, they also indicate that a substantial portion of the variation in competitive performance remains unexplained.

This outcome is not unexpected. Competitive performance depends on many factors that were intentionally outside the scope of this dataset, including:

- Map and game mode
- Team composition and synergy
- Matchup-specific interactions
- Balance patches over time
- Player skill distribution
- Pick and ban rates

Consequently, this model should not be interpreted as a complete predictor of the competitive meta. Instead, it measures how much predictive information is contained within a brawler's intrinsic design characteristics alone.

One of the most important findings of this project is that thoughtfully engineered gameplay features contain meaningful predictive signal, but they cannot fully explain competitive success without additional contextual information.

9. Discussion

9.1 Initial assumption

The early expectation was that choosing a stronger algorithm would be the main path to better predictions. The project showed otherwise. The largest jump came from moving beyond a linear baseline, but subsequent progress depended more on data quality, evaluation discipline, and feature design than on trying many model families.

9.2 Corrections made during development

Initial inefficiency or risk	Correction
Reliance on a single train/test split	Adopted five-fold cross-validation
Feature lists duplicated across scripts	Centralized definitions in <code>feature_groups.py</code>
Assumption that more variables help	Ran feature-group and leave-one-out ablations
Manual guess at reduced feature set	Used RFECV to select the subset systematically
Comparing models before fixing inputs	Benchmarked models only after selecting common features
Repository tracking Python cache files	Added <code>.gitignore</code> and removed cached artifacts

9.3 Most important engineering lesson

Representation before complexity

At the beginning of the project, I expected selecting a better machine learning algorithm to produce the largest performance gains. Instead, the most significant improvements came from improving the representation of the problem. Engineering meaningful gameplay features, validating the dataset, and adopting a more rigorous

evaluation strategy consistently produced larger gains than simply switching algorithms. This reinforced an important lesson in applied machine learning: model performance is often limited more by data quality and feature representation than by algorithm choice.

10. Limitations

- Sample size: 103 brawlers is small for comparing many engineered variables.
- Manual labels: gameplay scores contain informed but subjective judgments.
- Static snapshot: performance changes with balance patches and seasonal metas.
- Missing context: maps, modes, teammates, opponents, and matchup structure are absent.
- Target construction: MetaScoreScaling inherits assumptions from the underlying Meta Score.
- Evaluation variance: fold-level MAE variation remains substantial.
- No external holdout: the final model has not been tested on future releases.

These limitations explain why final R^2 remained modest. The model found useful signal, but most variance in competitive performance is likely driven by factors outside static brawler design or by interactions not represented in the dataset.

11. Recommended Next Phase

11.1 High-value data additions

9. Map- and mode-specific performance rather than one aggregate competitive profile.
10. Balance-patch history and time-series Meta Score.
11. Matchup and counter matrices.
12. Team-composition and synergy features.
13. Pick, ban, and player-skill distributions.
14. Future brawler releases as a true out-of-sample test set.

11.2 Modeling extensions

XGBoost would be a reasonable final algorithmic benchmark, but it is not the highest-priority next step. The evidence suggests that richer contextual data would provide more value than another round of model switching. Once longitudinal or matchup data are available, nested cross-validation, permutation importance, SHAP analysis, and temporal holdout evaluation would become appropriate.

12. Conclusion

The Brawl Stars Balance Analyzer developed from a simple analytics idea into a complete applied machine learning workflow. The project required defining a defensible target, integrating multiple data sources, engineering domain-specific features, building reproducible evaluation scripts, diagnosing failed assumptions, selecting a compact feature set, and benchmarking alternative models.

The final tuned Random Forest achieved the best consistent benchmark result with MAE 1.476 and R^2 0.137 using six gameplay features. More importantly, the project demonstrated that gameplay-oriented feature engineering carried more predictive signal than raw statistics and that model simplicity could improve without sacrificing performance.

The final model should be viewed as a strong baseline rather than a complete predictor of the live meta. Its value lies in both the result and the process: it establishes a reproducible framework that can be expanded as richer competitive data become available.

More than producing a predictive model, this project demonstrates an iterative engineering workflow: defining a measurable problem, constructing reliable datasets, designing domain-specific features, evaluating competing hypotheses, and refining the solution based on empirical evidence. Those principles extend beyond game analytics and are directly applicable to real-world machine learning systems.

Appendix A. Final Selected Features

Feature	Role in the model
SurvivabilityScore	Ability to remain effective under pressure
DecisionComplexity	Breadth and difficulty of meaningful choices
SkillCeiling	Potential reward from mastery
EscapeScore	Reliability of disengagement
MobilityScore	Ability to reposition and control engagement distance
PositioningImportance	Dependence on spatial decision-making

Appendix B. Final Random Forest Configuration

```
RandomForestRegressor(  
    n_estimators=500,  
    max_depth=2,  
    min_samples_split=10,  
    min_samples_leaf=1,  
    max_features='log2',  
    random_state=42  
)
```

Appendix C. Reproducibility Artifacts

- data/master_dataset.csv
- src/machine_learning/feature_groups.py
- src/machine_learning/random_forest_regression.py
- src/machine_learning/feature_ablation.py
- src/machine_learning/leave_one_feature_out.py
- src/machine_learning/rfecv_feature_selection.py
- src/machine_learning/model_benchmark.py
- src/visualization/
- results/
- charts/